Экскурс по проекту мультимодального определения контрафакта Ozon

Дата подготовки: 03.05.2026

1. Назначение проекта

Проект решает задачу бинарной классификации товарных карточек: определить, относится ли товар к контрафактным/проблемным по целевой метке resolution.

Главная идея проекта - использовать не один источник информации, а несколько модальностей:

- табличные признаки карточки, продаж, возвратов, рейтингов и продавца;
- текстовые поля карточки товара;
- изображение товара.

Такой подход выбран потому, что признаки контрафакта могут проявляться по-разному. Например, аномальные возвраты и продажи видны в числах, подозрительные формулировки - в описании товара, а визуальное сходство или качество упаковки - на изображении.

2. Какие данные используются

В проекте ожидается следующая структура данных:

- data/ml_ozon_counterfeit_data/ml_ozon_counterfeit_train.csv - обучающая таблица;
- data/ml_ozon_counterfeit_data/ml_ozon_counterfeit_test.csv - внешний test без целевой переменной;
- data/ml_ozon_counterfeit_train_images/*.png - изображения для train;
- data/ml_ozon_counterfeit_test_images/*.png - изображения для внешнего test.

Фактический размер train CSV:

- строк: 197 198;
- колонок: 45;
- положительный класс resolution=1: 13 052 строк;
- отрицательный класс resolution=0: 184 146 строк;
- доля положительного класса: около 6.62%.

Фактический размер внешнего test CSV:

- строк: 22 760;
- колонок: 44.

В train CSV есть 4 текстовые/идентификационные object-колонки:

- brand_name;
- description;
- name_rus;
- CommercialTypeName4;

и числовые признаки, например:

- счетчики рейтингов rating_1_count ... rating_5_count;
- счетчики комментариев, фото и видео;
- цена PriceDiscounted;
- возраст товара item_time_alive;
- продажи, возвраты и fake returns за окна 7/30/90 дней;
- GMV за 7/30/90 дней;
- признаки принятых и возвращенных экземпляров;
- ItemVarietyCount, ItemAvailableCount;
- seller_time_alive;
- идентификаторы ItemID, SellerID.

Целевая переменная:

- resolution.

3. Первичная загрузка и нормализация ID

Загрузка train выполняется функцией load_train_frame() из src/ozon_counterfeit/data.py.

На этом шаге:

1. CSV читается через pandas.read_csv(..., low_memory=False).
2. В начало таблицы добавляется служебный row_id.
3. ItemID приводится к строке.
4. SellerID приводится к строке.
5. resolution приводится к int.

Почему это важно:

- row_id нужен как стабильный ключ строки после merge, split и сохранения manifest;
- ItemID и SellerID приводятся к строкам, потому что это идентификаторы, а не числовые величины;
- строковый тип предотвращает ошибки merge, когда один источник читает ItemID как object, а другой как int64;
- SellerID используется для seller-aware split, поэтому его тип должен быть единым.

4. Manifest: связка таблицы, картинок и split

После загрузки строится manifest через build_train_manifest().

Manifest содержит:

- row_id;
- ItemID;
- SellerID;
- resolution;
- image_path;
- has_image;
- split.

Изображения связываются с товарами по правилу:

ItemID -> data/ml_ozon_counterfeit_train_images/{ItemID}.png

Фактическое покрытие изображениями:

- всего строк: 197 198;
- строк с изображением: 196 460;
- строк без изображения: 738.

Проверка exact_mapping_ok=true означает, что для строк с изображением имя файла действительно соответствует ItemID.

5. Разбиение train/val/test без seller leakage

Split строится функцией greedy_group_split() из src/ozon_counterfeit/splitting.py.

Используется group split по SellerID. Это значит, что все товары одного продавца попадают только в один из split:

- train;
- val;
- test.

Почему это сделано:

- если один продавец попадет и в train, и в test, модель может выучить особенности продавца, а не признаки контрафакта;
- это даст завышенную оценку качества;
- seller-aware split делает оценку ближе к реальному сценарию, где модель должна работать на новых или не виденных в обучении продавцах.

Фактическая проверка пересечений продавцов:

- train vs val: 0 общих продавцов;
- train vs test: 0 общих продавцов;
- val vs test: 0 общих продавцов.

Фактический split:

Split	Строк	Позитивов	Негативов	Доля positive	Уникальных продавцов	Строк с image
train	132 068	9 136	122 932	6.92%	11 073	132 014

| val | 32 516 | 1 958 | 30 558 | 6.02% | 34 | 31 883 |
| test | 32 614 | 1 958 | 30 656 | 6.00% | 60 | 32 563 |

6. Особенность сравнения экспериментов

В run_experiment() по умолчанию include_missing_images=False.

Поэтому для headline-экспериментов используется только подмножество строк с изображениями:

```
dataset = dataset.loc[dataset['has_image'] == 1].copy()
```

Это сделано, чтобы E0, E1 и E2 сравнивались на одном и том же наборе объектов. Иначе полная мультимодальная модель E2 работала бы только на товарах с картинками, а табличная модель могла бы обучаться и оцениваться на большем наборе, что делает сравнение менее честным.

7. Обработка табличных признаков

Табличные признаки обрабатываются в src/ozon_counterfeit/features.py.

7.1 Выбор числовых колонок

Функция infer_numeric_columns() берет числовые колонки, но исключает:

- resolution;
- id;
- ItemID;
- SellerID.

Причина:

- resolution - target, его нельзя подавать в признаки;
- id, ItemID, SellerID - идентификаторы, их прямое использование может привести к переобучению и leakage;
- модель должна учиться на содержательных свойствах товара и продавца, а не на произвольных номерах.

В текущем metadata для E0 в список numeric columns также попал row_id. Это служебный индекс строки. Его лучше считать техническим признаком и в дальнейшем исключить, чтобы снизить риск случайного позиционного leakage.

7.2 Логарифмирование

Функция infer_log_columns() выбирает признаки по ключевым словам:

- count;
- gmv;
- price;
- alive;
- value;
- accepted;
- returned;
- sales;
- returns;
- available;
- variety.

К ним применяется log1p.

Почему:

- счетчики продаж, возвратов и GMV обычно имеют тяжелые хвосты;
- несколько очень больших значений могут доминировать над моделью;
- log1p сжимает масштаб, сохраняя порядок значений;
- log1p корректно работает с нулями.

7.3 Пропуски

Для числовых признаков:

1. Создаются missing flags - бинарные индикаторы, где был пропуск.
2. Значения пропусков заполняются медианой через SimpleImputer(strategy="median").
3. Признаки стандартизируются через StandardScaler.

Почему:

- сам факт отсутствия значения может быть полезным сигналом;
- медиана устойчивее среднего к выбросам;
- стандартизация нужна нейросети, чтобы признаки разных масштабов были сопоставимы.

В текущем train самые заметные пропуски:

- счетчики рейтингов, комментариев, фото и видео: около 76.07%;
- brand_name: около 40.84%;
- description: около 13.22%;
- часть агрегатов по заказам, GMV и возвратам: около 4-5%.

8. Обработка текста

Текстовая модальность строится функцией build_text_corpus().

Используются поля:

- brand_name;
- name_rus;
- CommercialTypeName4;
- description.

Для каждой строки непустые поля склеиваются в один текст:

field_name: value [SEP] field_name: value ...

Если все поля пустые, используется заглушка:

description:

Почему:

- модель получает не только значение, но и название поля;
- [SEP] разделяет разные смысловые зоны карточки;
- пустые значения не ломают encoder;
- текстовый encoder может извлечь смысл бренда, названия, категории и описания.

Текст кодируется frozen-моделью:

sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

Эмбеддинги:

- считаются батчами;
- нормализуются (normalize_embeddings=True);
- сохраняются в .npy cache внутри директории эксперимента.

Почему frozen encoder:

- не нужно дообучать большую языковую модель;
- вычисления дешевле и стабильнее;
- можно один раз посчитать embeddings и переиспользовать.

9. Обработка изображений

Изображения используются в эксперименте E2.

Пайплайн:

1. Берется image_path из manifest.
2. Изображение открывается через PIL.
3. Приводится к RGB.
4. Применяются transforms, которые timm подбирает для конкретной модели.
5. Изображение пропускается через pretrained CNN без классификационной головы.
6. Получается image embedding.
7. Эмбеддинги сохраняются в .npy cache.

Используемая модель:

efficientnet_b0

Почему:

- EfficientNet B0 - компактная pretrained CNN;
- она дает хороший компромисс между качеством и скоростью;
- pretrained encoder позволяет извлекать визуальные признаки без обучения CNN с нуля;
- в задаче с большим числом изображений это практичнее, чем обучать визуальную часть end-to-end.

10. Модальности и modality flags

Функция build_modality_flags() создает три флага:

- image present;
- text present;
- tabular present.

Для E1 image embeddings заменяются нулями, а флаг изображения равен 0.

Для E2 image embeddings реальные, а флаг изображения равен 1.

Почему:

- модель явно знает, какая модальность присутствует;
- нулевой embedding без флага мог бы быть неоднозначным;
- такой подход позволяет одной архитектуре работать с разным набором модальностей.

11. Какие модели используются

11.1 E0: Random Forest на табличных признаках

E0 - базовая табличная модель:

```
RandomForestClassifier(
    n_estimators=400,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced_subsample",
)
```

Почему Random Forest:

- устойчив к нелинейным зависимостям;
- хорошо работает как сильный baseline на табличных данных;
- требует меньше настройки, чем градиентный бустинг или нейросеть;
- умеет оценивать важность признаков;
- class_weight="balanced_subsample" частично компенсирует дисбаланс классов.

Фактический результат уже сохраненного E0 на test:

- PR-AUC: 0.22495;
- precision: 0.32325;
- recall: 0.31119;
- F1: 0.31710;
- threshold: 0.1025.

Confusion matrix E0 на test:

```
| | Pred 0 | Pred 1 |
| True 0 | 29 331 | 1 275 |
| True 1 | 1 348 | 609 |
```

11.2 E1: Text + Tab Early Fusion

E1 использует:

- табличные признаки после preprocessing;

- текстовые embeddings SentenceTransformer;
- нулевые image embeddings;
- modality flags.

Классификатор:

EarlyFusionClassifier из src/ozon_counterfeit/models.py.

На момент подготовки отчета для E1 найден только сохраненный tabular_preprocessor.pkl; полноценные metrics.json и модель не сохранены. Это соответствует ранее возникшей ошибке отсутствующего sentence_transformers, после которой зависимость была установлена.

11.3 E2: Image + Text + Tab Early Fusion

E2 использует:

- табличные признаки;
- текстовые embeddings;
- image embeddings EfficientNet B0;
- modality flags.

Это полный мультимодальный эксперимент.

На момент подготовки отчета завершенных артефактов E2 в artifacts/experiments/e2 не найдено.

12. Архитектура EarlyFusionClassifier

Модель состоит из трех отдельных projection-блоков:

- tabular_projector;
- text_projector;
- image_projector.

Каждый блок:

1. Linear(input_dim, projection_dim);
2. LayerNorm;
3. ReLU;
4. Dropout.

После этого:

1. три projected-вектора склеиваются;
2. к ним добавляются 3 modality flags;
3. итоговый вектор идет в MLP:

- Linear(fused_dim, hidden_dim);
- ReLU;
- Dropout;
- Linear(hidden_dim, 1).

Выход - один logit для бинарной классификации.

Почему early fusion:

- модель может учить взаимодействия между таблицей, текстом и изображением;
- проще, чем сложные late-fusion ансамбли;
- хорошо подходит для уже готовых frozen embeddings.

13. Обучение нейросети

Обучение выполняется функцией train_early_fusion() из src/ozon_counterfeit/training.py.

Основные настройки:

- epochs: 12;
- batch size: 256;
- learning rate: 1e-3;
- weight decay: 1e-4;

- patience: 3;
- projection dim: 256;
- hidden dim: 256;
- dropout: 0.2;
- optimizer: AdamW;
- loss: BCEWithLogitsLoss.

Для борьбы с дисбалансом классов используется pos_weight:

negative_count / positive_count

Почему:

- положительный класс редкий, около 6.62%;
- без компенсации модель может предпочитать почти всегда предсказывать 0;
- pos_weight увеличивает штраф за ошибки на положительном классе.

Early stopping идет по validation PR-AUC.

14. Метрики и выбор threshold

Модель сначала выдает score/probability. Затем threshold выбирается на validation по максимальному F1.

Сохраняются метрики:

- pr_auc;
- precision;
- recall;
- f1;
- threshold;
- confusion_matrix;
- classification_report.

Почему PR-AUC важен:

- классы сильно несбалансированы;
- ROC-AUC при таком дисбалансе может выглядеть слишком оптимистично;
- PR-AUC лучше отражает качество поиска редкого положительного класса.

Почему threshold выбирается отдельно:

- стандартный порог 0.5 не оптимален при редком positive-классе;
- для E0 выбранный порог составил 0.1025, что заметно ниже 0.5;
- это позволяет поднять recall по положительному классу.

15. Кеширование артефактов

Проект сохраняет промежуточные и итоговые результаты в artifacts/.

Основные артефакты:

- artifacts/manifests/train_manifest.csv;
- artifacts/manifests/train_manifest_summary.json;
- artifacts/reports/data_checks.json;
- artifacts/experiments/e0/model.pkl;
- artifacts/experiments/e0/metrics.json;
- artifacts/experiments/e0/feature_importances.json;
- artifacts/experiments/<experiment>/cache/*.npy для embeddings;
- artifacts/experiments/<experiment>/metadata.json.

Почему это полезно:

- embeddings считаются долго, особенно image embeddings;
- можно перезапускать обучение без повторного кодирования текста и изображений;
- все параметры и метрики эксперимента остаются воспроизводимыми.

16. Внешний inference

В ноутбуке предусмотрен optional external inference:

- загружается ml_ozon_counterfeit_test.csv;
- строится manifest для test images;
- применяются сохраненные preprocessing и model weights;
- сохраняется файл предсказаний:

artifacts/predictions/external_test_predictions.csv.

По умолчанию:

RUN_EXTERNAL_INFERENCE = False

17. Итоговая логика проекта

Проект построен как последовательная ablation-схема:

1. E0 отвечает на вопрос: насколько хорошо работают только табличные признаки.
2. E1 проверяет, добавляет ли текст карточки полезный сигнал.
3. E2 проверяет, добавляют ли изображения дополнительный сигнал к таблице и тексту.

Такой дизайн удобен для дипломной работы, потому что он не просто обучает одну модель, а показывает вклад каждой модальности.

18. Рекомендации по улучшению

1. Исключить row_id из табличных признаков.

Это технический индекс, который не должен участвовать в обучении.

2. Завершить обучение E1 и E2.

Сейчас полностью сохранены метрики только для E0.

3. Добавить итоговую ablation table.

После обучения всех экспериментов удобно сравнить E0, E1, E2 по PR-AUC, precision, recall и F1.

4. Рассмотреть CatBoost/LightGBM как дополнительный табличный baseline.

Для табличных данных бустинг часто сильнее Random Forest.

5. Для Colab-запуска использовать GPU.

Это особенно важно для E2, где нужно кодировать изображения.